

Figure 3.17 The browser adds the Origin header before sending the request to the server.

to be the same origin. But remember that the origin comparison only compares the string values of the scheme, host, and port, and knows nothing about what host an IP address maps to. In this example, "localhost" and "127.0.0.1" are different strings, and therefore the request isn't a same-origin request. Now that you know what an origin is, let's look at how the browser sets the Origin header on requests.

3.4.3 Setting the Origin header

The browser adds the Origin header to the HTTP request before sending the request to the server. The browser is solely responsible for setting the Origin header. The Origin header is always present on cross-origin requests, and the client has no way of setting or overriding the value. This is a requirement from a security standpoint: if the client could change the Origin header, they could pretend to be someone they aren't. Figure 3.17 shows how the browser adds the Origin header before sending the request to the server.

Same-origin requests may sometimes have an Origin header as well. Chrome and Safari include an Origin header on same-origin non-GET requests. In these cases, the Origin header has the same value as the server's origin value. This is important to keep in mind. When identifying CORS requests, it's not enough to check that the Origin header exists. You should also check that the origin value is different from your server's origin value.

In this section you learned what an Origin header is, where you can find it, and how it can be used to identify a CORS request. Next, let's update the server to respond to the CORS request by using the Access-Control-Allow-Origin response header.

3.5 Responding to a CORS request

Look back at the conversation between the browser and the server in figure 3.13. The Origin header got you to the first part of the conversation, where the browser identifies the client. Now let's turn our attention to the second part of the conversation, where the server responds to the browser. The server does this by adding the Access-Control-Allow-Origin header to the response. Let's take a look at how this header works.

3.5.1 The Access-Control-Allow-Origin header

The server uses the Access-Control-Allow-Origin response header to approve the request. This header must be present on every successful CORS response. It completes